AI ENGINEERING RESEARCH INTERNSHIP | ALPHATRON

CampusPilot

An Adaptive Academic Recovery Agent

Technical documentation: RAG, study planning, quizzes and a LangGraph recovery agent with human approval

FastAPI | React + Vite | LangGraph | Gemini | RAG

PREPARED BY

Muzammil Ahmed

AI Engineering Research Intern, Alphatron

Version 1.0 | November 1, 2026

github.com/muzammilahmed321/Alphatron_Case_Study_project

About this document

Item	Detail
Project	CampusPilot, an adaptive academic recovery agent
Author	Muzammil Ahmed
Role	AI Engineering Research Intern, Alphatron
Version and date	1.0, November 1, 2026
Repository	github.com/muzammilahmed321/Alphatron_Case_Study_project
Stack	React + Vite, FastAPI, LangGraph, Google Gemini, sentence-transformers, NumPy
Current state	Working prototype. Data is kept in memory. PostgreSQL, login and deployment are planned.

Reviewer's quick path
Read chapter 0 (executive summary), then look at Figure 1 (architecture) and Figure 7 (recovery agent). Chapter 11 shows how the project was tested and what the results were, including the failures. Chapter 13 lists what is not finished.

Contents

0 Executive summary . . . 3
Project status at a glance . . . 3
Skills demonstrated . . . 3

1 Problem statement and objectives . . . 5
1.1 The problem . . . 5
1.2 Objectives . . . 5
1.3 How the original proposal feedback was used . . . 5

2 Solution overview . . . 5
A student's journey . . . 6

3 System architecture . . . 6
3.1 Technology choices . . . 6
3.2 Design principles . . . 7

4 Module 1: Course Knowledge Assistant . . . 7
4.1 Settings that matter . . . 8
4.2 Grounding behaviour . . . 8

5 Module 2: Study Planner . . . 8
5.1 Worked example . . . 9
5.2 Validator rules . . . 9

6 Module 3: Quizzes and progress analysis . . . 10
6.1 Classifying every topic . . . 10

7 Module 4: The Recovery Agent . . . 11
7.1 Nodes . . . 11
7.2 Routes . . . 12
7.3 Why the approval step is trustworthy . . . 12
8 Data model . . . 12
9 Frontend . . . 13
9.1 Pages . . . 14
10 API reference . . . 15
11 Testing and evaluation . . . 15
11.1 Criteria from the proposal . . . 15
11.2 How the evaluation runs went . . . 16
11.3 Latency and cost . . . 16
11.4 Manual test plan . . . 17
12 Problems encountered and how they were solved . . . 18
13 Limitations and roadmap . . . 18
13.1 Current limitations . . . 18
13.2 Roadmap . . . 18
14 How to run the project . . . 19
14.1 Requirements . . . 19
14.2 Configure . . . 19
14.3 Start the backend . . . 19
14.4 Start the frontend . . . 20
14.5 Run the automated evaluation . . . 20
15 Conclusion and learning outcomes . . . 20
What I learned . . . 20
Appendix A Sample requests . . . 20
Appendix B Glossary . . . 21
Appendix C References and acknowledgements . . . 21

0 Executive summary

CampusPilot is a study assistant that does more than answer questions. A student uploads course PDFs and enters exam dates. The system answers questions using only those documents, builds a day-by-day study plan, writes practice quizzes from the same notes, and works out which topics are weak. When tasks are missed or quiz scores drop, a LangGraph agent proposes a revised plan, explains the change, and **waits for the student to approve it**. Nothing in the plan changes without that approval.

The guiding idea is simple: **code does the maths, the language model does the language.** Scheduling, plan checking, quiz grading and the weak-topic rules are ordinary Python, so they are exact and testable. Gemini is used only to answer from retrieved text, write quiz questions and explain changes in plain words.

4 modules	**31** API endpoints	**8** UI pages	**10** agent graph nodes
25 automated checks	**13** manual test cases	**4** agent routes	**All** checks passed in final run

Project status at a glance

Component	Status	Notes
Course assistant (RAG)	Done	PDF upload, chunking, local embeddings, grounded and cited answers
Study planner	Done	Greedy scheduler, validator, honest warnings when time is short
Quizzes and progress	Done	Quiz generation from PDFs, exact grading, weak-topic rules
Recovery agent	Done	LangGraph routing, validation loop, human approval, checkpointing
React interface	Done	8 pages, edit and delete, history, animated dark UI
Evaluation	Done	Automated script, manual test plan, usage and cost tracking
PostgreSQL + pgvector	Planned	Storage is in memory today, so data resets when the server restarts
Login and per-user data	Planned	One shared workspace for now
Unit tests and deployment	Planned	Evaluation scripts exist, but no unit tests yet

Skills demonstrated

Skill	Where it shows up	Evidence in the repository
LangGraph	Recovery agent	Shared state, 10 nodes, conditional edges, retry loop, interrupt, checkpointer (backend/app/agents)
RAG	Course assistant	Parser, chunker, MiniLM embeddings, cosine search, grounded prompt (services/ingestion, services/rag)
LLM integration	Gemini calls	Grounded answers, JSON quiz generation, explanations, retry on 503 and 429, token and cost tracking
Backend API design	FastAPI	31 endpoints in 10 routers, request validation, clear error messages
Data modelling	Store layer	Six entities behind one interface so PostgreSQL can replace it
Frontend	React + Vite	8 pages, shared components, hand-written animated CSS

Skill	Where it shows up	Evidence in the repository
Evaluation	Testing	run_eval.py, a manual test plan, a record of failures and fixes

1 Problem statement and objectives

1.1 The problem

Students who juggle several courses usually have enough material: lecture notes, slides and past papers. What they lack is structure. They cannot easily see which topics are weak, how much time each one needs, or what to do after a bad week. A timetable made in September rarely survives November. General chatbots explain concepts well, but they do not know the exam dates, the study history or the last quiz result.

1.2 Objectives

#	Objective	Status
1	Answer questions only from uploaded PDFs, with page citations	Done
2	Generate a feasible study schedule from exam dates, daily hours and priorities	Done
3	Record quiz results and task completion, and find topics that need revision	Done
4	Detect missed targets or weak performance, revise the plan and explain the change	Done
5	Keep the student in control: no plan change without approval	Done
6	Measure accuracy, grounding, feasibility, routing, persistence of updates, latency and cost	Done
7	Keep data in PostgreSQL so progress survives restarts	Planned

1.3 How the original proposal feedback was used

The proposal asked four questions: is the problem meaningful, is the workflow agentic enough, is the scope right, and should the project go ahead. The answers shaped the build in these ways.

Feedback point	What was done
Make the recovery behaviour the centre	The Recovery Agent got the most design effort: routing, retry loop and approval step
Do not trust the model with scheduling	The planner and validator are plain Python. Gemini never does arithmetic
Show real tool use and a validation loop	The agent retrieves PDF pages, builds a plan, validates it and retries once if needed
Build evaluation early	A test PDF and an automated script were added before the interface
Control the scope	Modules 1 to 3 were built first and solidly, then Module 4 on top of their data

2 Solution overview

CampusPilot has four modules. Each one feeds the next: the assistant provides material, the planner provides tasks, quizzes and tasks provide progress data, and the agent uses all of it.

Module	What it does	Key technique
1 Course Knowledge Assistant	Upload PDFs, ask questions, get answers with file and page citations. Out-of-scope questions are refused.	RAG: chunks, embeddings, cosine search, grounded prompt
2 Personalised Study Planner	Turns exam dates, topics, priorities and daily hours into a schedule. Warns when time is not enough.	Greedy scheduler plus an independent validator
3 Academic Progress Analysis	Generates quizzes from the notes, grades them exactly, labels topics weak, untested or on track.	LLM writes questions, code grades them
4 Adaptive Recovery Agent	Finds weak topics or missed tasks, rebuilds the plan, explains it, waits for approval.	LangGraph: conditional routing, interrupt, checkpoints

A student's journey

Step	What the student does	What the system does
1	Uploads a course PDF	Reads the pages, splits them into chunks, stores their embeddings
2	Adds a course, exam date and topics	Stores them and lists them with priority and hours
3	Clicks Generate plan	Builds a schedule, validates it, saves tasks as pending
4	Takes quizzes and marks tasks	Grades answers, records results, updates each topic's status
5	Opens the Recovery Agent	Proposes a revised plan with an explanation and pages to review
6	Approves or rejects	Applies the plan if approved, keeps the old one if rejected

3 System architecture

The frontend only talks to the FastAPI routes. The routes are thin: they check the input, call a service or the agent, and return JSON. The real work lives in the services. All data access goes through one store class, so the storage can change without touching the routes.

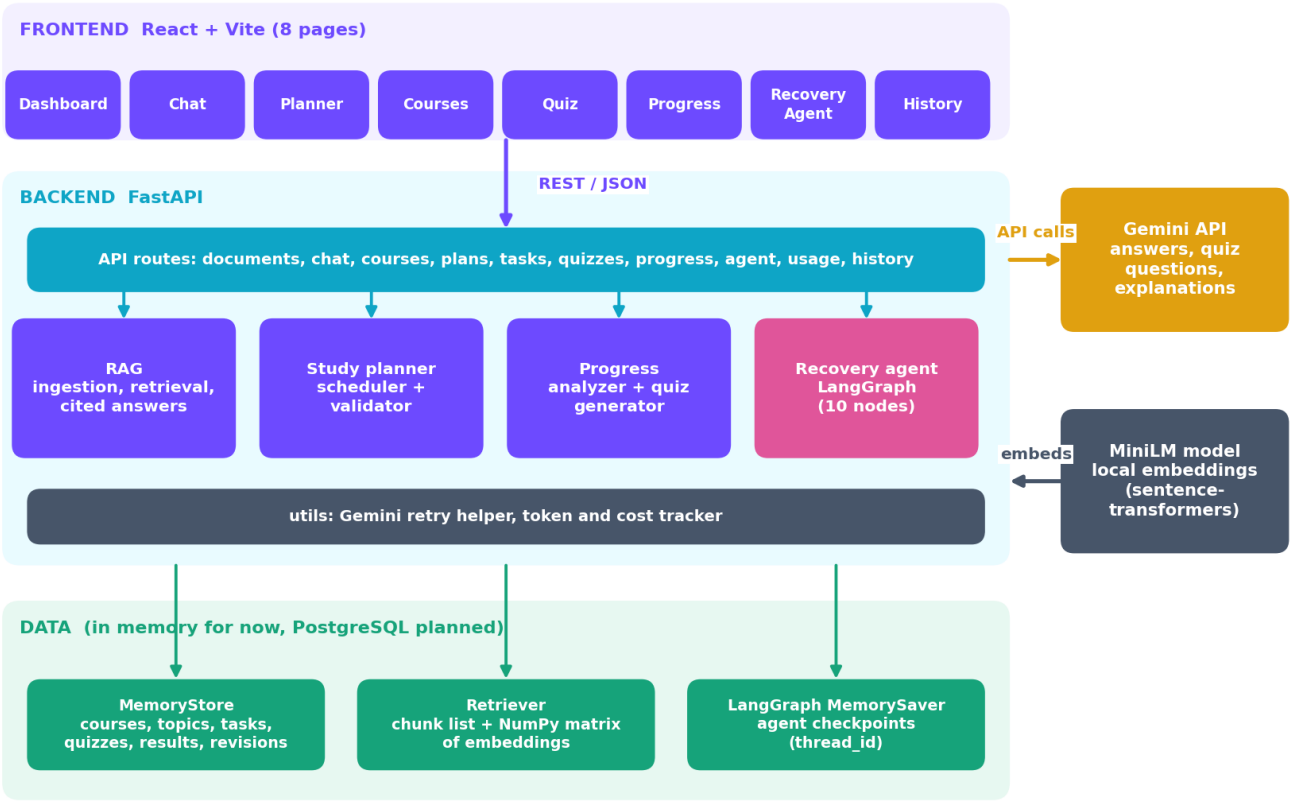


Figure 1: System architecture. The data layer is in memory today and is the part planned to move to PostgreSQL.

3.1 Technology choices

Layer	Technology	Why it was chosen
Frontend	React + Vite, plain CSS	Fast setup. Hand-written CSS keeps the animated design free of UI libraries
Backend	FastAPI, Pydantic	Request validation and an automatic /docs page that made testing every route easy

Layer	Technology	Why it was chosen
Agent	LangGraph	Shared state, conditional edges, pause and resume, checkpoints
Language model	Google Gemini (set in .env)	Free tier for development and usage metadata for cost tracking. The model name is a setting
Embeddings	all-MiniLM-L6-v2	Runs locally, no key, small download, 384-number vectors
Vector search	NumPy dot product	With normalised vectors this equals cosine similarity. Fine for thousands of chunks
PDF reading	pypdf	Simple and pure Python
Storage	In-memory store	Chosen on purpose so the whole pipeline could be tested before schema design

3.2 Design principles

Code does the maths
Anything that can be computed is computed in code: schedules, plan validation, quiz grades and weak-topic rules. The model is used only for language, so results are exact and failures are easy to explain.

Human in the loop
The agent only proposes. A revision stays pending until the student presses Approve. Reject leaves the active plan untouched.

Replaceable storage
Every read and write goes through one MemoryStore class with plain method names. A PostgreSQL version with the same method names can replace it without changing any route.

4 Module 1: Course Knowledge Assistant

Uploaded PDFs are split into chunks and turned into embeddings once. Each question is embedded the same way, compared with every chunk, and the best matches are sent to Gemini together with strict instructions to answer only from them.

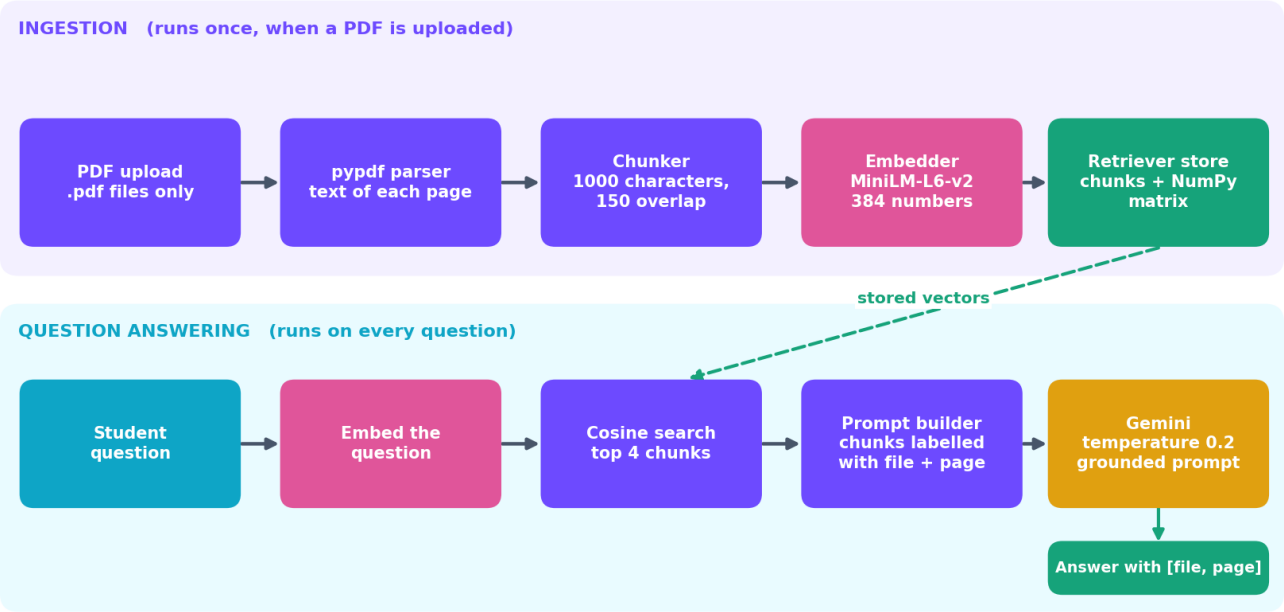


Figure 2: The two lanes of the RAG pipeline. Ingestion runs once per upload, question answering runs on every question.

4.1 Settings that matter

Setting	Value	Reason
Chunk size	1000 characters	Large enough for context, small enough to stay focused
Overlap	150 characters	A sentence cut at a boundary still appears whole in a neighbouring chunk
Embedding model	all-MiniLM-L6-v2 (384 numbers)	Local, free, good general semantic quality
Chunks per question	4 (adjustable per request)	Enough context without a long prompt
Temperature	0.2	Keeps answers close to the source text
Citation format	[filename, page X]	Every chunk keeps its page number from the parser
Re-uploading a file	Replaces the old copy	No duplicate chunks

4.2 Grounding behaviour

Question type	Expected behaviour	Verified by
In the PDF (for example, what is a primary key)	Answer built from retrieved text, with page citations	Keyword and citation checks
Not in the PDF (for example, capital of France)	Says it could not find the answer in the uploaded documents	Refusal check
No document uploaded	Clear message: upload a document first	Manual test
Gemini busy or rate limited	Retries with a growing delay, then a readable error	Evaluation runs

Known limits
Text-based PDFs only (scanned pages are skipped, and an all-scanned file is rejected). Fixed-size chunks can cut a word in half. Retrieval is purely semantic and shares one search space across all documents.

5 Module 2: Study Planner

The planner is a greedy algorithm chosen because it is easy to explain and easy to check. It sorts topics by exam date and priority, then fills each day up to the daily limit, giving no topic more than 2 hours a day.

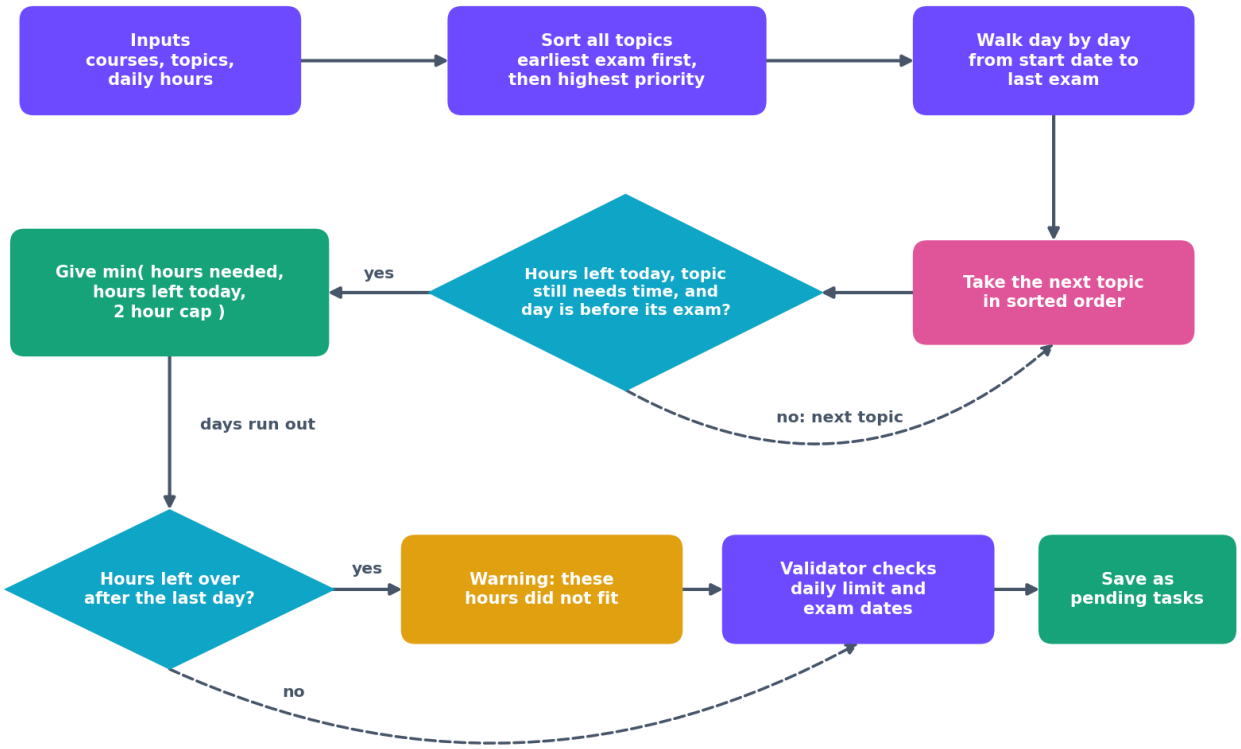


Figure 3: How the study schedule is built, and how leftover hours are reported.

5.1 Worked example

Course Databases, exam in 14 days, 3 study hours a day. Topics: Normalization (high, 4h), SQL Joins (medium, 3h), Transactions (medium, 2h) and Indexes (low, 2h). That is 11 hours in total.

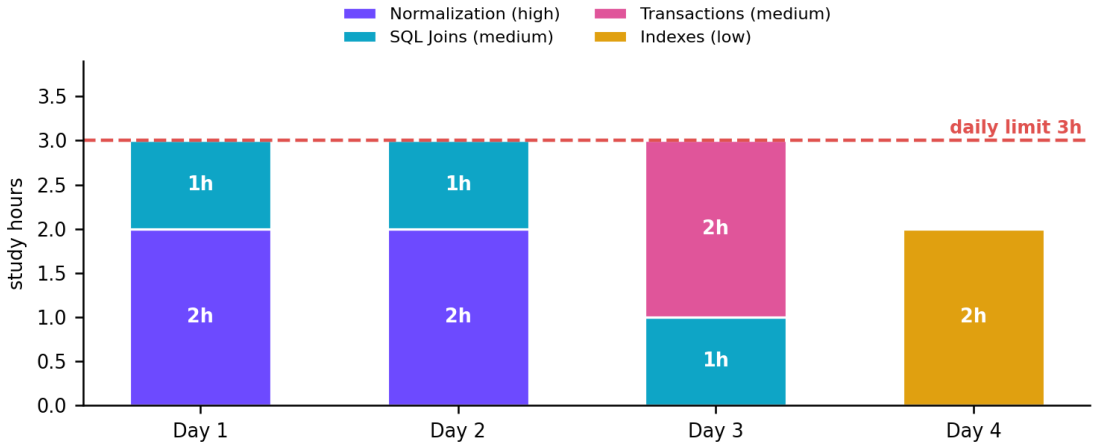


Figure 4: The generated schedule for the worked example. No day goes over the 3 hour limit.

5.2 Validator rules

Rule	Checked how	If broken
No day exceeds the daily limit	Hours are summed per date	The plan is refused instead of saved
No task on or after its exam date	Each task date is compared with its course exam date	The plan is refused instead of saved
Not enough time	Leftover hours after the last day	A warning names the topic and the hours that did not fit

> **Honest warnings**
> With 0.5 hours a day and 11 hours of work, the planner schedules what fits and reports the rest. A plan that looks fine but cannot be followed is worse than an honest warning.

Task handling. Saving a new plan replaces only pending tasks, so done and missed tasks stay as history. Tasks can be marked done or missed, and pending tasks dated before today are detected as missed. The completion rate is done tasks divided by done plus missed.

6 Module 3: Quizzes and progress analysis

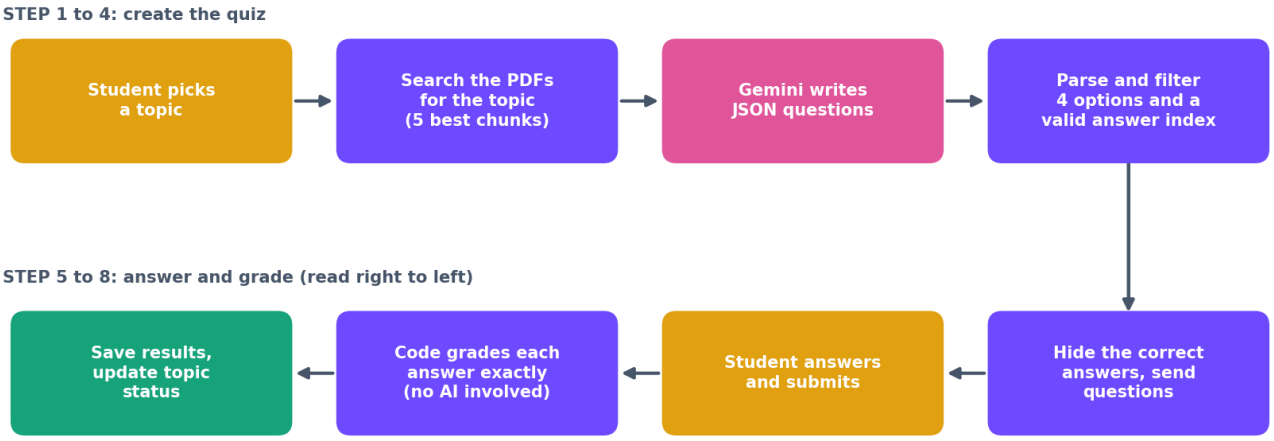


Figure 5: Quiz life cycle. The model writes questions, but the grading is done by code.

The correct answers are stored on the server and are not sent to the browser until the quiz is submitted. A quiz can be submitted once. Questions that do not have exactly four options and a valid answer index are dropped before the quiz is stored.

6.1 Classifying every topic

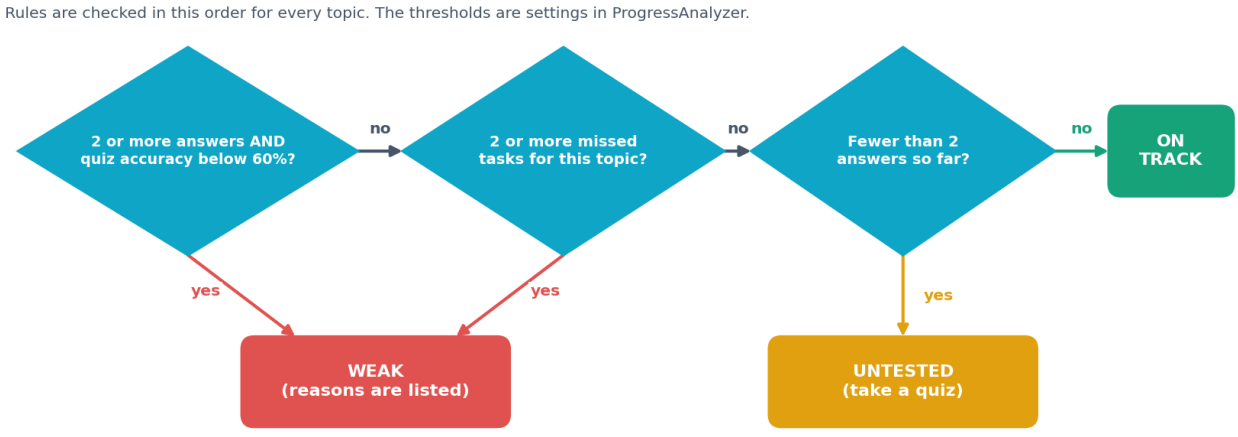


Figure 6: The rules that label a topic as weak, untested or on track.

Threshold	Default	Effect
Quiz accuracy below	60%	Weak, once enough answers exist
Minimum answers before judging	2	Fewer answers means untested
Missed tasks at or above	2	Weak, regardless of quiz data

Weak topics are listed first in the progress report, each with the reason (for example, quiz accuracy is 33 percent). The thresholds are constructor arguments, so they are easy to change and easy to discuss in an evaluation.

7 Module 4: The Recovery Agent

The recovery behaviour is a workflow with state and branches: read data, choose a path, build something, check it, possibly retry, then stop and wait for a person. That is what LangGraph is designed for. The agent keeps its state under a thread id, so it can pause between two API requests and resume later.

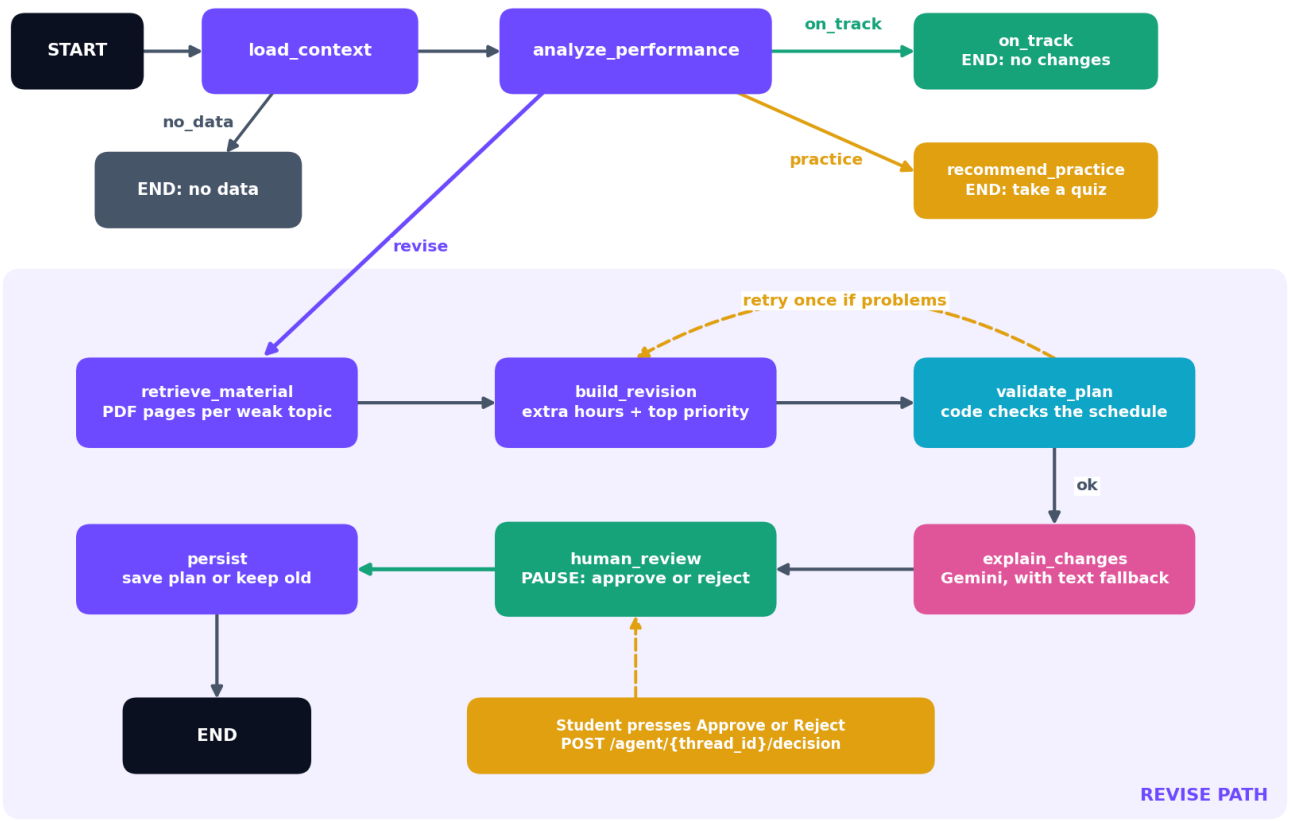


Figure 7: The recovery agent graph. The revise path ends with a human review step that pauses the workflow.

7.1 Nodes

Node	What it does
load_context	Reads saved data, marks overdue pending tasks as missed, stops early if there are no topics
analyze_performance	Runs the progress analyzer and picks the route
retrieve_material	Finds two relevant PDF chunks per weak topic, so the student knows which pages to review
build_revision	Builds a new schedule: weak topics get top priority and one extra hour, stored as a pending revision
validate_plan	Checks the schedule with the validator. If there are problems and fewer than two attempts, goes back to build_revision (without the extra hours)
explain_changes	Gemini turns the facts into 3 or 4 friendly sentences. A plain-text fallback is used if the call fails
human_review	Calls LangGraph interrupt and waits for approve or reject

Node	What it does
persist	On approve the proposed tasks become the active plan and weak topics keep their new priority and hours. On reject nothing changes
recommend_practice	Ends the run with the topics that need a quiz
on_track	Ends the run with a message that nothing needs to change

7.2 Routes

Route	Triggered when	Result
no_data	No courses or topics exist	Stops with a message to add them first
practice	No weak topics, but some topics have too little quiz data	Lists the topics to quiz
on_track	Every topic is healthy	Stops with no changes
revise	At least one weak topic (accuracy under 60% or 2+ missed tasks)	Builds, validates and explains a new plan, then waits for approval

7.3 Why the approval step is trustworthy

Safeguard	How it works
Proposal, not change	The new plan is stored as a pending revision. The active plan is untouched until Approve
Independent check	A separate validator checks daily limits and exam dates before the plan is even shown
One retry	If the first plan does not fit, a second plan is built without the extra hours. Remaining warnings are shown to the student
Visible reasons	The student sees the weak topics, the pages to review, any warnings and the full new schedule
Reversible	Reject leaves the current plan exactly as it was

Example run
A student has four Databases topics and a saved plan. Two Normalization quizzes are answered mostly wrong, so Normalization turns weak. The agent finds the Normalization pages in the PDF, builds a plan with extra Normalization time, validates it and shows the explanation and schedule. The Planner page is still unchanged. Pressing Approve replaces the pending tasks. Pressing Reject keeps the old ones.

8 Data model

All records live in one store class. Chunks and embeddings are kept separately by the retriever. Identifiers are short random hex strings.

Retriever (separate, in memory): chunk list + matrix of embeddings

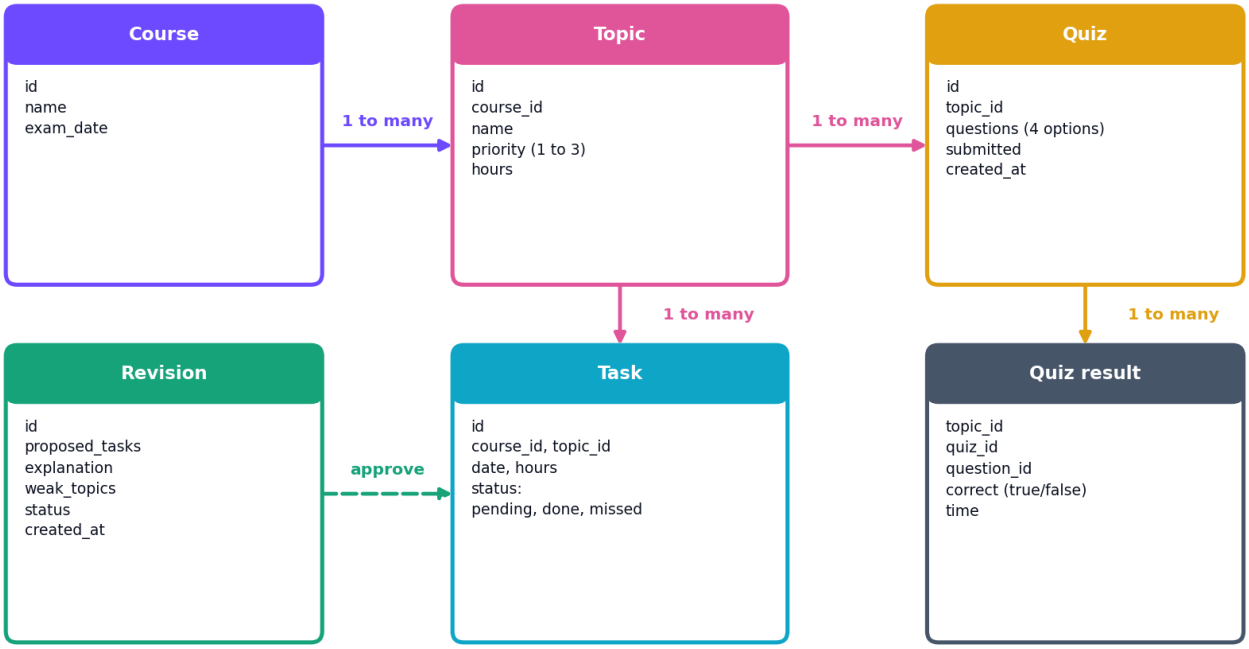
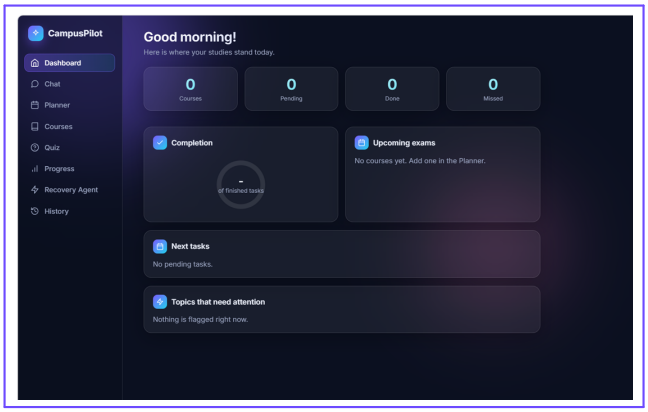


Figure 8: Entities and how they relate. A revision becomes active tasks only when it is approved.

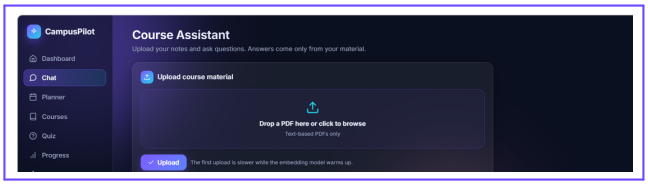
Consistency rule	Why it exists
Saving a plan replaces only pending tasks	Done and missed tasks are the student's history
Deleting a topic also deletes its tasks, quizzes and results	Avoids orphaned records
Deleting a course deletes everything under it	Same reason, one level up
A revision can be changed only while pending	An approved or rejected proposal must not be altered by a later agent run
A quiz can be submitted once	Prevents re-scoring and inflated accuracy

9 Frontend

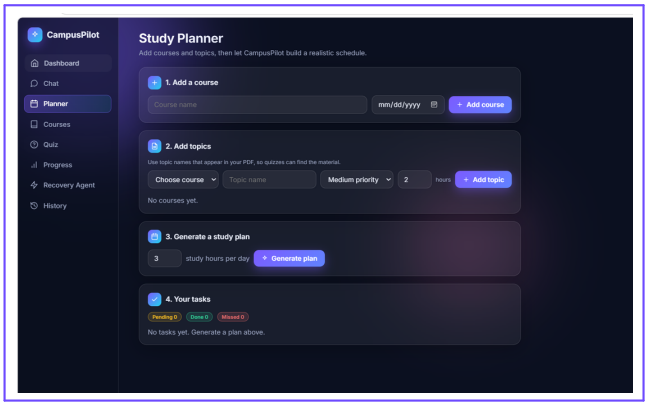
The interface is a single React page with a sidebar, eight screens and a shared set of small components. It uses a dark glass style with slowly moving gradient blobs, cards that fade in, an animated progress ring, chat bubbles with a typing indicator, quiz options that turn green or red, and a pulsing agent pipeline.



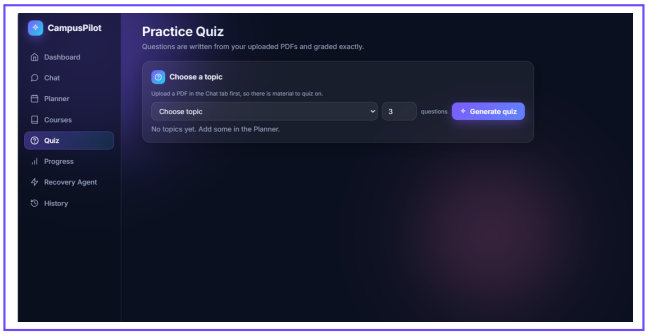
Dashboard



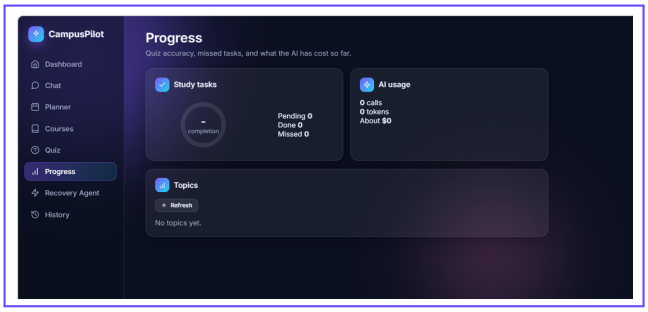
Course Assistant (upload)



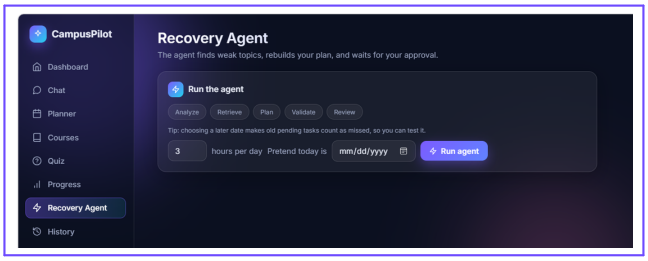
Study Planner



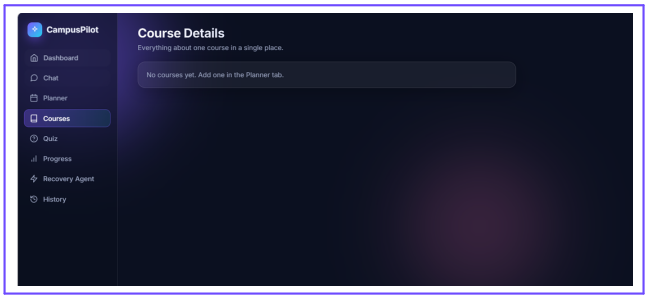
Practice Quiz



Progress



Recovery Agent



Course Details

Screenshots were taken at the start of testing, before any data was entered, so the screens show their empty states. Populated versions are in the repository's docs folder.

9.1 Pages

Page	Purpose
Dashboard	Counters, completion ring, upcoming exams with days left, next tasks, topics that need attention
Chat	Drag-and-drop PDF upload, document list with delete, chat thread with source chips, tokens and seconds
Planner	Add, edit and delete courses and topics, generate a plan, mark tasks done, missed or delete them
Courses	One course in detail: exam countdown, hours still planned, completion ring, topic bars, tasks
Quiz	Choose a topic, generate a quiz, answer, submit and see corrections with explanations
Progress	Task summary, accuracy bars with status badges, AI usage and cost
Recovery Agent	Run the agent, read the proposal, approve or reject
History	Past quizzes, agent revisions with their status, finished tasks

Implementation notes. A small API client provides three helpers (request, send JSON, delete) and reads the backend address from an environment variable. Errors from the API are shown in red notices using the message the server returned. The layout collapses to a top bar on narrow screens, and animations switch off for users who prefer reduced motion.

10 API reference

The backend exposes 31 endpoints in 10 routers, plus a health check. FastAPI generates an interactive reference at /docs.

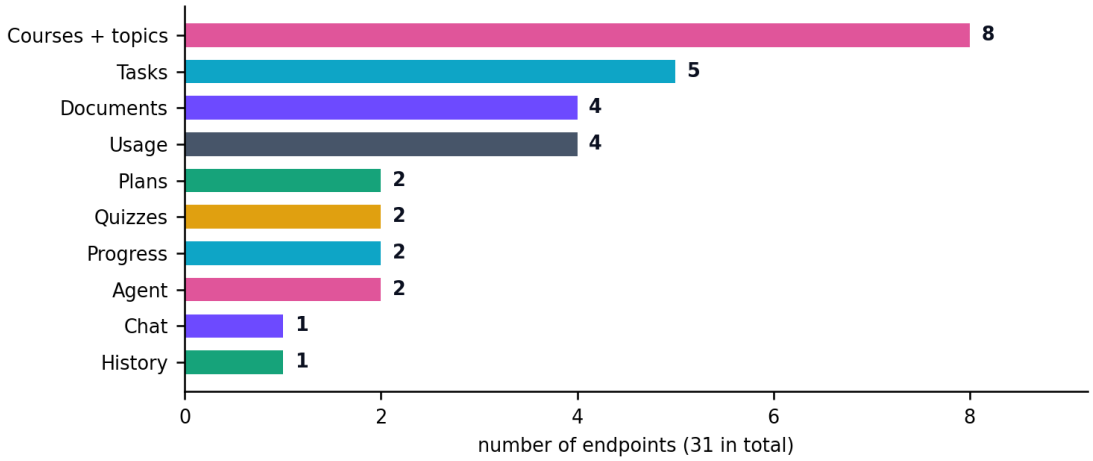


Figure 9: Number of endpoints per area.

Area	Endpoints
Documents	POST /documents/upload, GET /documents, DELETE /documents/{filename}, GET /documents/search
Chat	POST /chat/ask
Courses and topics	POST, GET /courses; PATCH, DELETE /courses/{id}; POST, GET /courses/{id}/topics; PATCH, DELETE /courses/{id}/topics/{topic_id}
Plans	POST /plans/generate, POST /plans/generate-and-save
Tasks	GET /tasks, GET /tasks/summary, PATCH, DELETE /tasks/{id}, POST /tasks/check-missed
Quizzes	POST /quizzes/generate, POST /quizzes/{id}/submit
Progress	GET /progress/topics, GET /progress/weak-topics
Agent	POST /agent/run, POST /agent/{thread_id}/decision
Insights	GET /history, GET /usage, GET /usage/calls
Testing helpers	POST /usage/reset, POST /usage/reset-all-data (to be removed before deployment)

11 Testing and evaluation

11.1 Criteria from the proposal

Proposal criterion	How it is checked	Final run
Answer accuracy and relevance	Keyword checks on answers about a sample PDF	Passed
Grounding	Out-of-scope questions must be refused	Passed
Citations	In-scope answers must mention a page	Passed
Schedule feasibility	No rule violations on a valid plan; a warning on an impossible one	Passed
Agent routing	no_data, practice and revise are each triggered	Passed

Proposal criterion	How it is checked	Final run
Persistence of updates	Nothing applied while waiting; approve saves; reject keeps the old plan	Passed
Latency and cost	Tokens, seconds and estimated cost per feature from /usage	Recorded

11.2 How the evaluation runs went

The failures in the early runs were useful. They were not logic errors in the system. They came from a missing route, a retired model name, and the Gemini API being busy or rate limited. Each one led to a fix that is described in chapter 12, and the final run passed every check.

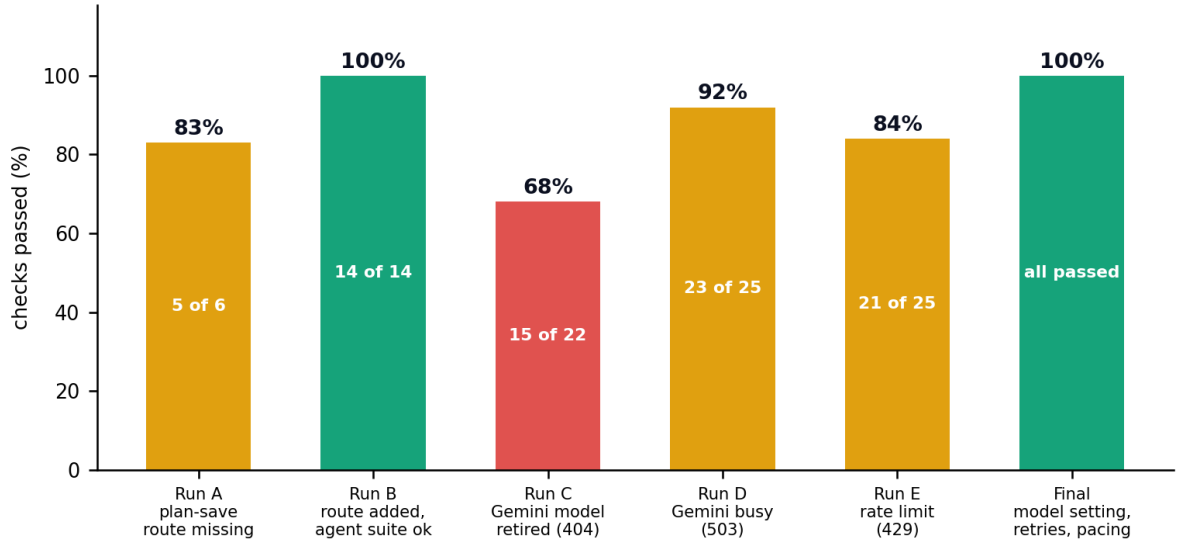


Figure 10: Share of checks passed in each evaluation run, with the cause of each shortfall. The first four runs had fewer checks because the RAG section was skipped until a test PDF was added.

11.3 Latency and cost

The figures below come from one earlier run with Gemini 3.5 Flash: 6 calls, 3,314 input tokens and 308 output tokens.

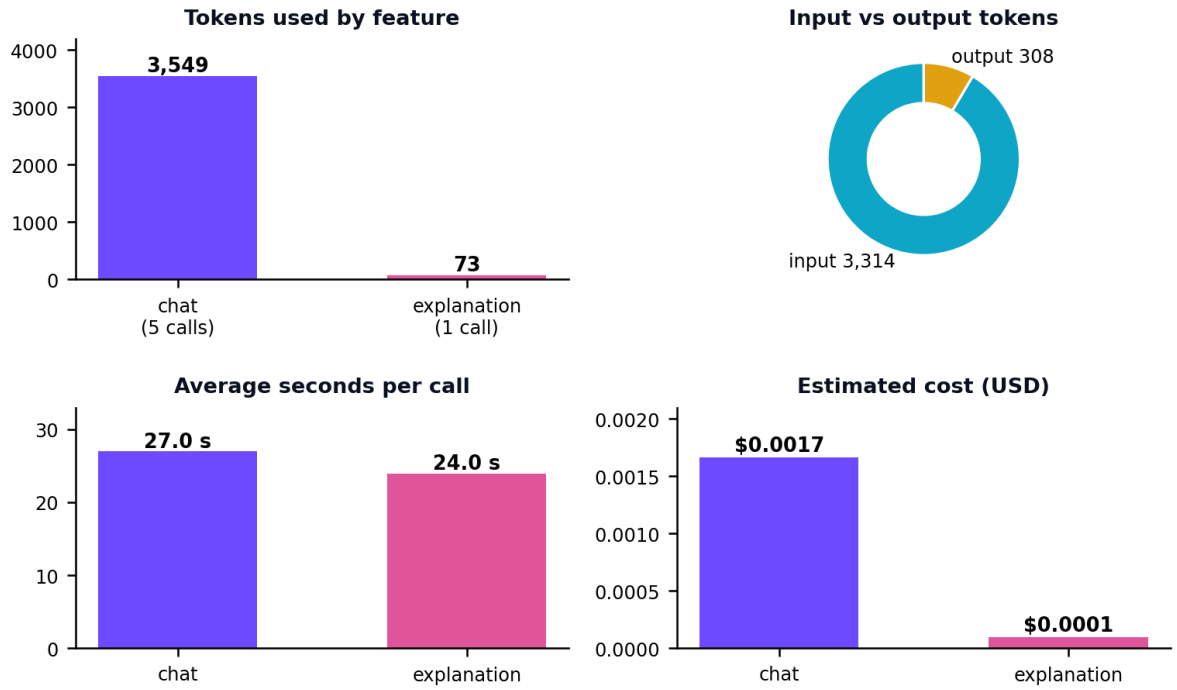


Figure 11: Token use, time per call and estimated cost in one run.

Read these numbers carefully
The response times include time spent retrying while the service was busy, so they overstate normal speed, but even so about 25 seconds is slow for a chat feature. The cost figures use per-token prices configured for an earlier model and not yet updated, so treat them as rough estimates and check the current price list before quoting them.

11.4 Manual test plan

A step-by-step plan runs through the web interface in order, because later tests depend on data created by earlier ones. Test data was a four-page PDF about databases and a course called Databases with four topics totalling eleven hours.

Test	Area	What it covers
1	Empty state	All pages load, the agent returns no_data
2	Upload	Document list, replacing a file, rejecting non-PDF files
3	Grounded answers	Four in-scope and two out-of-scope questions
4	Courses and topics	Validation of empty names and zero hours
5	Plan and feasibility	Daily limit, 11 scheduled hours, warning at 0.5 hours a day
6	Task status	Done, missed, delete, completion ring
7	Edit and delete	Rename, change hours, confirm or cancel deletion
8	Quiz and grading	Generate, submit, corrections shown
9	Progress	Weak and untested labels, usage figures
10	Agent routes	Revise with approve, reject, missed tasks, practice, on track
11	History	Quizzes, revisions, finished tasks
12	Error handling	Backend stopped, no document uploaded
13	Automated evaluation	run_eval.py end to end

What is not tested
There are no unit tests yet. Retrieval quality was checked with one short PDF and a handful of questions, and the accuracy check is keyword matching, which is crude. There was no load testing and no automated browser testing.

12 Problems encountered and how they were solved

Problem	What happened	Fix	Outcome
Empty scaffold files	A script created the whole folder tree first. Blank files made imports fail again and again	Filled each file, and used a one-line check that lists empty files	Fixed
Docker friction	Docker Desktop was not running and was not worth the effort	Switched to a Python virtual environment; the database moved to later	Mitigated
Retired Gemini model	Every model call returned 404 after the model name was retired	Model name moved to a setting read from one place	Fixed
Silent fallback	The agent explanation quietly used plain text, hiding the failure	Errors are now printed to the server log	Fixed
503 and 429 responses	Gemini was sometimes busy or rate limited	Retry helper with growing delay; evaluation pacing; skipped instead of failed	Mitigated
A test failing for the wrong reason	An agent check failed because the save-plan route was missing, not because of the agent	Checks now print the route, message and task counts	Fixed
Slow responses	Calls took about 25 seconds in some runs	Not solved. Ideas: lighter model, shorter prompts, streaming	Not fixed
Secret handling	An API key briefly ended up in a source file	Moved to .env, which is git-ignored. Exposed keys should be replaced	Fixed

13 Limitations and roadmap

13.1 Current limitations

Limitation	Impact	Detail
Data is in memory	High	Courses, tasks, quizzes, revisions, chunks and agent checkpoints are lost when the backend restarts
No user accounts	High	One shared workspace, and the API has no authentication
Slow model calls	Medium	Answers, quizzes and explanations can take 10 to 30 seconds
Text-based PDFs only	Medium	Scanned documents are rejected, and tables or diagrams are not understood
Simple chunking and search	Medium	Fixed-size chunks, one search space for all documents
Greedy planner	Low	Front-loads work, no rest days or spaced revision
Estimated costs	Low	Prices in the tracker must be updated for the model in use

13.2 Roadmap

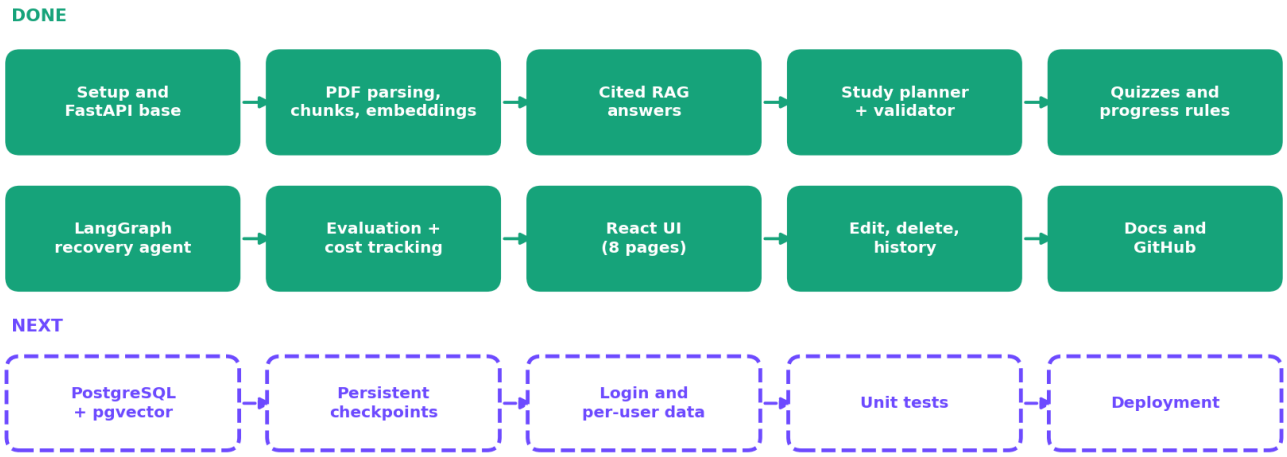


Figure 12: What has been completed (solid) and what comes next (dashed).

Next step	Why it matters
PostgreSQL with pgvector and Alembic migrations	Progress must survive restarts, which is the core of the proposal
Persistent agent checkpoints	A paused agent should survive a restart
Login and per-user data	Needed before the app can be shared
Unit tests for planner, analyzer and routing	Plain Python functions that are easy to test and catch regressions
Deployment with a hosted database	Makes the project usable by others
Better retrieval and planning	Paragraph-aware chunks, per-course search, rest days and spaced revision

14 How to run the project

14.1 Requirements

- Python 3.11 and Node.js 20 or newer
- A Google Gemini API key from Google AI Studio
- About 1 GB of disk space, mostly for PyTorch and the embedding model

14.2 Configure

Create a file named .env in the campuspilot folder (it is git-ignored):

```
GEMINI_API_KEY=your-key-here
GEMINI_MODEL=gemini-3.5-flash
```

14.3 Start the backend

```
git clone https://github.com/muzammilahmed321/Alphatron_Case_Study_project.git
cd Alphatron_Case_Study_project/campuspilot/backend
python -m venv .venv
.\.venv\Scripts\Activate.ps1        # macOS/Linux: source .venv/bin/activate
pip install -r requirements.txt
uvicorn app.main:app --port 8000
```

Wait for the line Application startup complete. Run it without --reload, because data is in memory and auto-reload would wipe it on every file save. The first start downloads the embedding model (about 90 MB).

14.4 Start the frontend

```
cd campuspilot/frontend
npm install
npm run dev
```

Open http://localhost:5173. The API pages are at http://localhost:8000/docs.

14.5 Run the automated evaluation

```
python ..\evaluation\run_eval.py     # from the backend folder, with the server running
```

Setting	Where	Meaning
GEMINI_API_KEY	campuspilot/.env	Required. Never commit it
GEMINI_MODEL	campuspilot/.env	Optional. Defaults to gemini-3.5-flash. Change it if the model is retired
VITE_API_URL	frontend/.env	Optional. Backend address, defaults to http://localhost:8000

Symptom	Likely cause	Fix
Failed to fetch	Backend not running	Start it on port 8000
Chat shows Not Found on load	Old backend code still running	Restart the backend
Upload a document first	Backend restarted, memory cleared	Upload the PDF again
Gemini error 404 about the model	Model name retired	Set GEMINI_MODEL to a current model
Gemini error 503 or 429	Service busy or free-tier limit	Wait a minute and try again

15 Conclusion and learning outcomes

CampusPilot does what the proposal set out to do, with one honest exception. A student can upload notes, ask questions that are answered only from those notes, get a feasible study plan, take quizzes that are graded exactly, and let an agent propose a recovery plan that changes nothing until it is approved. The recovery agent shows the LangGraph features that were planned: shared state, conditional routing, a validation loop, a human review step and checkpointing.

The design choice I would defend most is keeping arithmetic and grading in code and using the language model only for language. It made the system easier to test and made failures easier to explain. The first thing I would fix is storage: an assistant that forgets everything when the server restarts cannot yet claim to follow a student's progress over time. PostgreSQL, persistent checkpoints and login are the next steps.

What I learned

- How to structure an agent workflow as a graph, and why a pause-and-resume step changes the design.
- How to make a language model reliable by limiting what it is allowed to do, and by checking its output in code.
- That evaluation finds real problems: a retired model, rate limits and a missing route were all discovered by tests, not by guessing.
- That failures should explain themselves: error messages and test details that name the cause save hours.
- How to keep a project small enough to finish by building modules in an order where each one feeds the next.

Appendix A Sample requests

Endpoint	Body
POST /chat/ask	`{"question": "What does First Normal Form require?", "top_k": 4}`
POST /courses	`{"name": "Databases", "exam_date": "2026-11-15"}`
POST /courses/{id}/topics	`{"name": "Normalization", "priority": 3, "hours": 4}`
POST /plans/generate-and-save	`{"daily_hours": 3}`
POST /agent/run	`{"daily_hours": 3, "today": "2026-11-08"}`
POST /agent/{thread_id}/decision	`{"approve": true}`

Appendix B Glossary

Term	Meaning
RAG	Retrieval-augmented generation: find relevant passages first, then ask the model to answer from them
Chunk	A short piece of a document, kept with its page number, that is embedded and searched
Embedding	A list of numbers representing the meaning of a text, so similar texts get similar numbers
Cosine similarity	How closely two embeddings point in the same direction
Grounding	Keeping an answer tied to the supplied documents instead of the model's general knowledge
LangGraph	A library for building stateful workflows as graphs of steps with conditional edges
Interrupt	A LangGraph feature that pauses a workflow until outside input arrives
Checkpointer	Storage for a workflow's state so it can be resumed later
Revision	A proposed plan change waiting for approval
Weak topic	Quiz accuracy under 60% (with at least 2 answers) or at least 2 missed tasks

Appendix C References and acknowledgements

- Lewis et al., 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.
- Reimers and Gurevych, 2019. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks.
- Wang et al., 2020. MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers.
- Official documentation of FastAPI, LangGraph, React, Vite, sentence-transformers, pypdf and the Google Gemini API.

Thanks to Alphatron for the opportunity to work on this project as part of the AI engineering research internship, and for the feedback on the original proposal.